AI Frontier: An Interactive Dashboard for Mapping the LLM Landscape

EECE 5642 — Data Visualization | Final Project Report

Avo Sarkissian

Department of Electrical and Computer Engineering | Northeastern University | Spring 2026

sarkissian.a@northeastern.edu

Abstract

Hundreds of large language models are now publicly available through hosted APIs, each with different tradeoffs in price, speed, and capability, but there's no single place to compare them across all those dimensions at once. AI Frontier is an interactive dashboard tracking 255+ models across 29 providers, pulling live pricing and benchmark data every hour from the Artificial Analysis API. It covers text, image, and video generation models alongside a hardware compatibility tool for locally runnable open-weight models. The system runs in Python using Plotly Dash and has been live since March 2026. This report covers the data pipeline, composite intelligence scoring, and the design decisions behind each of the eleven visualization tabs, with particular attention to the Run Local feature.

1. Introduction

A year ago, picking an LLM for a project meant choosing from maybe five or six realistic options. That's not the case anymore. As of spring 2026, there are commercially hosted models from OpenAI, Anthropic, Google, Meta, Mistral, Cohere, and dozens of others, each with different pricing tiers, context windows, and benchmark profiles. A model scoring 70 on reasoning benchmarks might cost 20 times more per token than one scoring 65. Some tasks don't need the smartest model available; they need the fastest or cheapest one that clears a minimum quality bar. Without a unified place to see this, choosing a model is mostly trial and error.

Existing tools each solve part of the problem. The LMSYS Chatbot Arena has good quality coverage but doesn't show pricing. The Hugging Face Open LLM Leaderboard is useful for open-weight models but doesn't include hosted API services. Provider pricing pages list costs but not benchmark scores. And none of them address local hardware compatibility: if you want to know whether Llama 3.3 70B will run on your GPU at a useful speed, you're cross-referencing multiple spec sheets and doing arithmetic by hand.

AI Frontier started from that specific frustration. I kept needing to have three or four tabs open simultaneously to make even basic model selection decisions, and none of the existing tools covered the local hardware question at all. The dashboard pulls live price and performance data on an hourly schedule and organizes it into eleven views, each built

around a question users actually ask. It's been deployed at a public URL and all the code is open-source at https://github.com/Avo-Sarkissian/AI-Frontier-Database. The live dashboard is at https://ai-frontier-database.onrender.com.

2. Data Sources and Pipeline

2.1 Cloud Model Data

The main data source is the Artificial Analysis API, which aggregates pricing and benchmark results for commercially hosted LLMs. For each model it returns price per million tokens (input and output, at the cheapest available host), median throughput in tokens per second, median time-to-first-token, context window size, and a composite quality score. The scraper runs as a background thread that fires once an hour, starting when the app launches.

Every successful response gets written to a live CSV cache and also saved as a timestamped snapshot. By mid-April 2026 over 30 daily snapshots had accumulated, which is what powers the Trends tab. The price field shown throughout the dashboard is a blended rate computed from raw input/output prices at a 3:1 output-to-input ratio, a rough approximation of real-world token usage, but better than showing two separate numbers on every chart.

2.2 Local Model and Hardware Data

The local model catalog is a separate, manually maintained dataset. GPU specs (VRAM capacity and memory bandwidth) came from official product pages for NVIDIA, AMD, Apple, and Intel hardware. Model parameter counts and quantization support came from Hugging Face model cards. This one doesn't update automatically; I update it when a notable new GPU generation or model family ships. Hardware specs don't change the way API prices do, so a static catalog is the right call here.

3. Intelligence Score and Benchmark Methodology

Rather than building a custom benchmark aggregation, the dashboard uses the composite intelligence score that Artificial Analysis computes and exposes through their API. It's a 0–100 score averaged across four categories:

- **Reasoning:** MMLU-Pro and GPQA, testing factual knowledge and multi-step deductive reasoning.
- **Coding:** LiveCodeBench and SciCode, evaluating writing and debugging real programs.
- **Math:** AIME and MATH-500, covering competition-style quantitative problems.
- **Knowledge:** Humanity's Last Exam, a recently released benchmark of expert-level questions across scientific domains.

The rationale for averaging across categories is contamination resistance. A model that was heavily fine-tuned on MMLU-style data might top that individual test while

being mediocre on code or math. An average across four independent evaluations is harder to game and gives a more reliable picture of general capability. It also means API and open-weight models end up on the same scale, which is important for the Run Local comparison.

The downside of any composite score is that it collapses real differences. Two models at 72 can have completely different profiles — one strong at reasoning and weak at math, the other the reverse. That's exactly what the Compare tab is for: it breaks out all four dimensions on a radar chart so you can see where models actually differ rather than just comparing single numbers.

4. Dashboard Design

4.1 Tab Structure

The dashboard has eleven tabs, each built around a specific question:

- **Overview:** Bubble scatter of all 255+ models with a Pareto frontier overlay. X-axis is log-scaled price, y-axis is intelligence score, bubble size encodes throughput.
- **Agent Stack:** Recommends a three-tier model configuration (reasoning, balanced, fast) for agentic workflows, filtered by available hardware.
- **Landscape:** Treemap of the AI industry: tile area encodes model count per provider, color intensity encodes average intelligence score.
- **Rankings:** Top 25 models sorted by intelligence, value (score per dollar), or speed, with tier-separator lines at meaningful performance gaps.
- **Compare:** Radar chart for up to five models across five dimensions: intelligence, speed, affordability, context window, and latency.
- **Budget:** Estimates projected monthly API cost given a user-supplied token volume, sorted cheapest-first.
- **Table:** Full sortable table of all 255+ models with every available metric.
- **Run Local:** VRAM compatibility calculator and inference speed estimator for open-weight models, filtered by GPU and quantization level.
- **Image Gen:** ELO-ranked image generation model leaderboard from the AA Image Arena.
- **Video Gen:** Ranked comparison of video generation models with pricing and quality data.
- **Trends:** Price-over-time line chart built from daily historical snapshots, showing how API pricing has shifted since February 2026.

4.2 Visualization Choices

Overview (bubble scatter with Pareto frontier). The problem with showing 255 models simultaneously is that a ranked list throws away the tradeoff information. A model ranked 5th might be half the price of the model ranked 4th with nearly identical quality. The bubble scatter puts price on the x-axis, quality on the y-axis, and encodes

throughput as bubble size, so three variables are visible at once. Price needs to be on a log scale: the spread goes from under a cent to over $15 per million tokens, and a linear axis would compress nearly everything into one corner.

The dotted Pareto frontier connects models that aren't dominated in the cost-quality tradeoff — points where you can't improve quality without paying more, or cut cost without accepting lower performance. It's recomputed from the live data on every load using a standard non-dominated sorting pass in Pandas. Provider colors and marker shapes are both encoded on the same dots, so colorblind users don't need a separate accessibility mode.

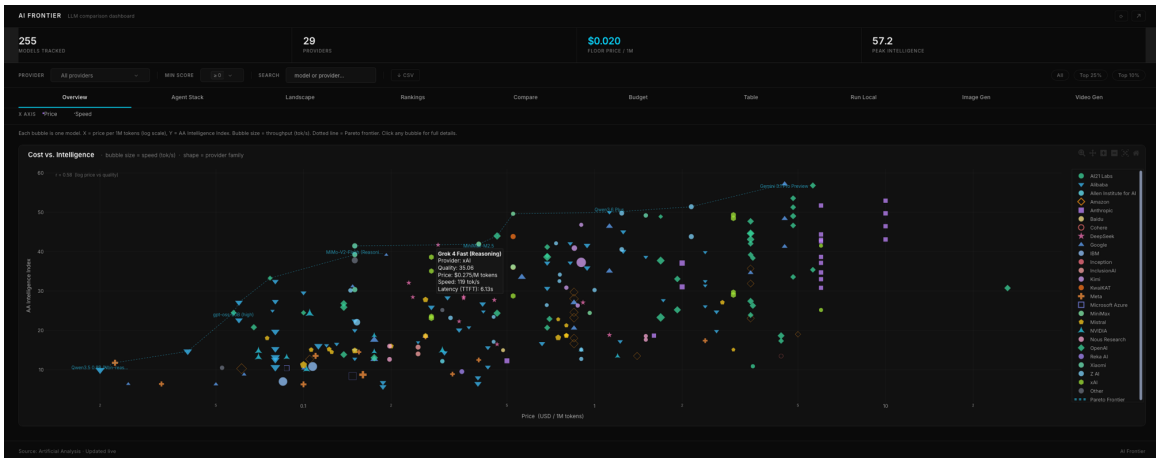

Figure 1. Overview tab: Cost vs. Intelligence bubble scatter with Pareto frontier overlay. Each dot is one model; bubble size encodes throughput. The dotted line connects undominated models in the cost-quality tradeoff.

Rankings (horizontal bars with tier separators). Rankings are one-dimensional comparisons, and horizontal bars handle that more clearly than a scatter with one wasted axis. Tier separator lines appear wherever the gap between adjacent models is large enough to represent a real break in performance, not just noise. This keeps users from treating a 72 and a 71 as equivalent just because they're next to each other on the list.

Compare (radar chart). When you want to see how five models differ across five dimensions at once, a radar chart is the right tool. Each axis is normalized 0–5. The known limitation is that the shape depends on axis ordering, which is fixed; rearranging the axes would change what the polygon looks like without changing any underlying data. It's still more useful than five separate bar charts for getting a quick cross-model profile.

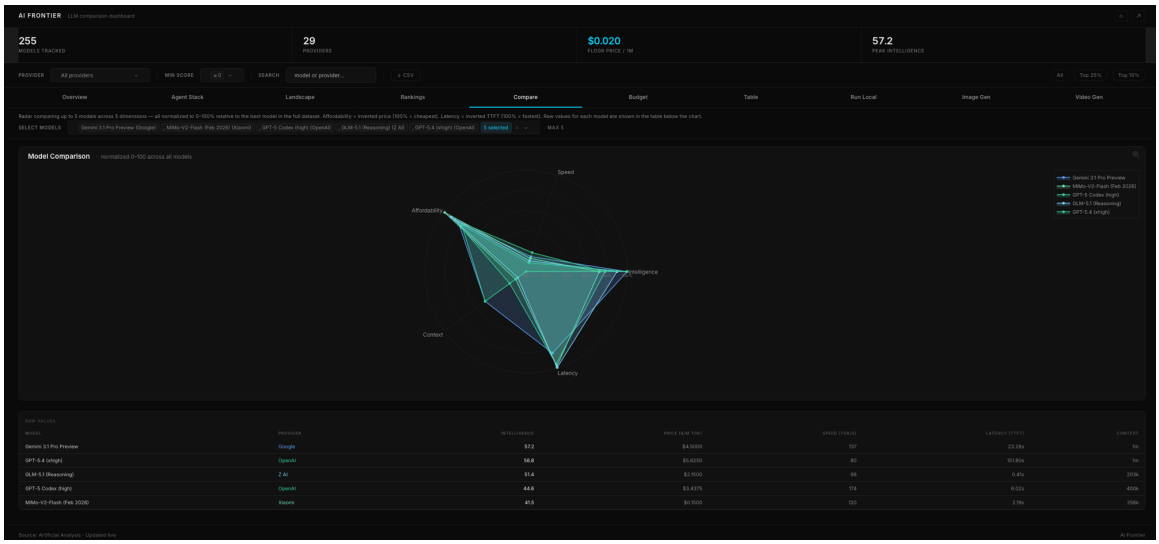

Figure 2. Compare tab: radar chart for five models across Speed, Affordability, Intelligence, Context, and Latency. Raw values shown in the table below the chart.

Landscape (treemap). Tile area encodes model count per provider; color intensity encodes average intelligence score. It answers a different question than the other tabs: who dominates by volume, which providers are newcomers with a narrow catalog, which ones specialize in high-capability models. A ranked bar chart sits directly below it for precise comparisons — the treemap gives the big picture, the bar chart gives the exact numbers.

Trends (line chart over time). This tab only works because the scraper saves timestamped snapshots rather than overwriting the same file. Looking at prices from February through April 2026 makes something obvious that a static dataset would hide completely: the market moved fast. Several major providers dropped prices significantly in that window.

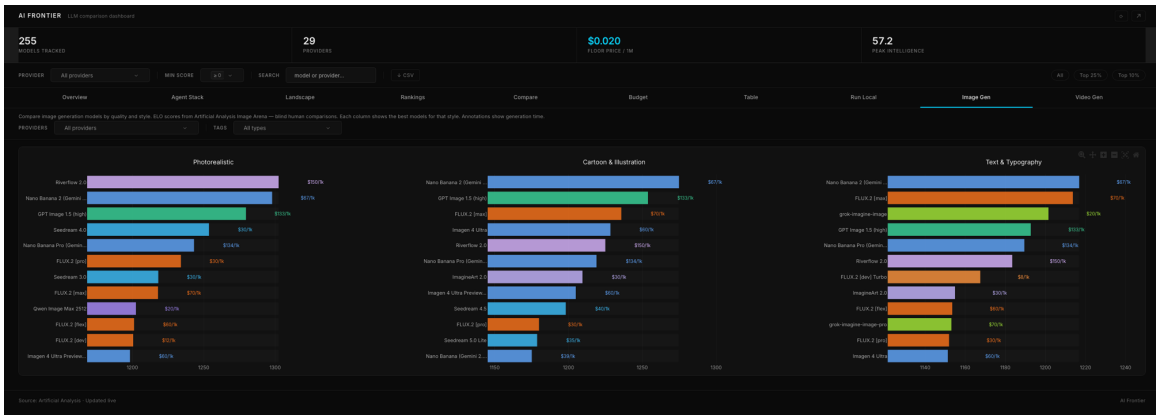

Figure 3. Image Gen tab: ELO-ranked image generation models broken out by style category. Bar length is ELO score from the AA Image Arena; price per 1k images annotated on each bar.

4.3 Run Local: Hardware Compatibility and Inference Estimation

The most technically involved tab to build was Run Local. The question it answers — which open-weight models will actually fit on my GPU, and how fast will they run — isn't something any major leaderboard addresses. Running models locally has specific advantages: data stays on the machine, there's no per-token cost after the hardware is paid for, there's no network latency, and it works offline. The complication is that the answer depends on parameter count, quantization level, and the GPU's VRAM and memory bandwidth all interacting together. Working it out manually means pulling specs from several product pages and doing arithmetic. The tab automates all of that.

Users pick a GPU from a dropdown covering NVIDIA RTX 3000, 4000, and 5000 series; AMD RX 7000 series; Apple Silicon from M1 through M4 (Pro, Max, and Ultra variants); and Intel Arc. Then they select a quantization level: FP16/BF16 for full precision, or Q8, Q4, Q2 for progressively smaller weights. Two outputs update immediately: a compatibility table of every model that fits in the selected GPU's VRAM, and a scatter plot of the full catalog with intelligence score on the x-axis and estimated tokens per second on the y-axis.

VRAM is estimated as $N \times B \times 1.2$, where N is parameter count in billions, B is bytes per weight (2 for FP16/BF16, 1 for Q8, 0.5 for Q4, 0.25 for Q2), and the 1.2 factor covers KV-cache, activations, and runtime overhead. Speed is estimated as memory bandwidth (GB/s) divided by $(N \times B)$. That formula comes from a straightforward observation: local LLM inference at single-user scale is almost always memory-bandwidth-bound rather than compute-bound. Every forward pass loads the model weights from memory; the GPU's ability to move data is the bottleneck, not its floating-point throughput. Double the bandwidth, roughly double the tokens per second.

Apple Silicon is worth calling out specifically. The M-series chips use unified memory, meaning the CPU, GPU, and neural engine all share the same physical pool. That makes the effective 'VRAM' the full system memory (96 GB on an M2 Max, 192 GB on an M2 Ultra), rather than a discrete chip with its own fixed capacity. Combined with memory bandwidth that's competitive with mid-range discrete GPUs, this makes Apple Silicon surprisingly capable for local inference. An M3 Max at 128 GB can run a 70B model at Q4 comfortably; on an RTX 4090 (24 GB), that same model needs Q2 just to fit. The dashboard uses Apple's published bandwidth figures, so the estimates reflect real hardware rather than guesses.

The scatter plot shows incompatible models in muted gray rather than hiding them. That decision matters: a model needing 26 GB on a 24 GB card shows up dimmed rather than absent, so the user can see it's close and try a lower quantization level without guessing. Compatible models are plotted in full color with marker size proportional to parameter count, making it easy to spot where large-but-slow models cluster versus the smaller-but-faster ones. Dropping from Q8 to Q4 cuts VRAM roughly in half and doubles speed, but may shave a few benchmark points depending on the model; the scatter makes that tradeoff visible.

Figure 4. Run Local tab: VRAM Requirement vs. Intelligence scatter (top) and ranked compatibility table (bottom) for an NVIDIA RTX 5090 at Q4. Compatible models are colored by family; incompatible models shown in gray.

Every open-weight model in the catalog uses the same composite intelligence score as the cloud models elsewhere in the dashboard. That means Llama 3.3 70B at Q4 on a local GPU and GPT-4o on a hosted API both appear on the same 0–100 axis. Tools like Ollama and LM Studio handle local inference well, but they don't put local models in context against the broader API landscape. That comparison is what makes Run Local more than just a VRAM calculator.

4.4 Design Principles

A few design choices applied across all eleven tabs. The biggest was data-ink ratio: every gridline, chart border, and legend entry that didn't encode actual data got removed. The dark background (#0a0a0a) helps here: dark backgrounds make data elements stand out without needing the heavy whitespace that a white-background chart at the same density would require.

Provider colors and marker shapes are both encoded on the same dots and bars, and they stay consistent across all eleven tabs. That came from a specific accessibility concern: someone with red-green colorblindness shouldn't need a separate mode to read the charts. Keeping the same shapes across tabs also means you only have to learn the legend once.

Price axes use log scale throughout, as do context window sizes (which range from 8K to over a million tokens). Linear scales for distributions this skewed compress all the variation into one end of the axis. Most models sit at the low end of both price and context ranges, which is exactly where log scale preserves the most detail.

4.5 Technical Stack

I built the whole stack in Python (data pipeline, scraper, and dashboard) using Plotly Dash for the web layer. Dash compiles Python component trees into a live web app without a JavaScript build step, which kept everything simple. No Node.js, no webpack, no separate API server. The whole thing starts with python app.py.

The app is hosted on Render, which redeploys automatically on every GitHub push. The main tradeoff with Dash versus a React frontend is that client-side interactions need a server round-trip. In practice that wasn't a problem: the dataset is under a megabyte, callbacks come back in roughly 200ms on the Render instance, and the use case doesn't require instant client-side updates the way a real-time trading dashboard would.

5. Contributions

The thing this project adds that existing tools don't have is live, multi-modal coverage in one place. Leaderboards like LMSYS and Open LLM are either human-preference ranked with no pricing, or benchmark-only with no API metadata. Provider pricing pages don't have quality scores. Nobody covers text, image, and video generation models alongside local hardware compatibility in a single interface that updates automatically.

The Agent Stack tab goes a step further than ranking: given a workflow type, it recommends a three-tier model configuration (reasoning, balanced, and fast) and lists specific models at each tier. For anyone building an agentic pipeline, that's a more direct answer than scrolling a ranked list and making the configuration decision themselves.

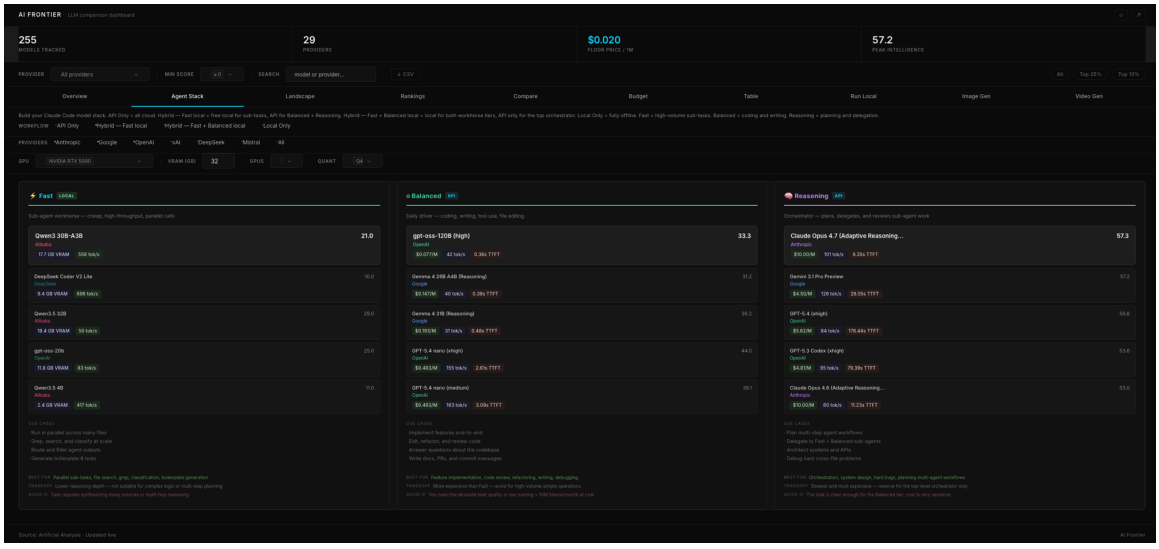

Figure 5. Agent Stack tab: three-tier model configuration for a Hybrid workflow. Fast tier runs locally; Balanced and Reasoning tiers use cloud APIs. Each card shows VRAM, speed, price, and latency for the recommended model.

All the code is open-source. The dashboard has been live since March 2026 with over 30 daily snapshots archived, meaning the Trends tab will only get more useful over time.

6. Future Work

The most useful thing I didn't get to is per-category score breakdowns in the Compare tab. Artificial Analysis provides reasoning, coding, math, and knowledge scores separately, but currently the dashboard only shows the composite. Surfacing those four dimensions on the radar chart would let users make much more targeted selections.

The Budget tab only estimates cost from raw token count. It doesn't account for prompt caching, which can cut effective cost by 80% or more for certain use cases, or for multi-turn context buildup. Both matter for real production applications. Fine-tuning cost estimation — weighing training compute against inference savings — is a related addition that would help teams decide whether to fine-tune a cheaper model or just pay for a better one.

Real measured latency by region would also be valuable. Published time-to-first-token medians from provider spec sheets don't always match what you actually see, especially under load. The hourly scraper is already running; adding a synthetic request layer would just mean logging actual response times alongside the existing metadata. A mobile layout is also on the list; most tabs are built for a wide desktop viewport and would need real work on smaller screens.

7. Conclusion

Building this project made a few things clear that weren't before. The LLM market moves fast enough that a static tool is basically outdated the week it's published: prices dropped meaningfully for several providers just in the time between starting this project and finishing it. The Run Local comparison ended up mattering more than expected: there are open-weight models that are competitive with paid APIs and run comfortably on consumer hardware, but you'd never know it without something that puts them on the same scale. The dashboard is still running at the GitHub link below and will keep updating as long as the Artificial Analysis API does.

References

[1] Artificial Analysis. Artificial Analysis API: LLM performance and pricing benchmarks. https://artificialanalysis.ai, 2024.

[2] W.-L. Chiang et al. Chatbot Arena: An open platform for evaluating LLMs by human preference. arXiv preprint arXiv:2403.04132, 2024.

[3] Hugging Face. Open LLM Leaderboard. https://huggingface.co/spaces/HuggingFaceH4/open_llm_leaderboard, 2023.

[4] Y. Wang et al. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. arXiv preprint arXiv:2406.01574, 2024.

[5] D. Rein et al. GPQA: A graduate-level google-proof Q&A benchmark. arXiv preprint arXiv:2311.12022, 2023.

[6] N. Jain et al. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. arXiv preprint arXiv:2403.07974, 2024.

[7] M. Tian et al. SciCode: A research coding benchmark curated by scientists. arXiv preprint arXiv:2407.13168, 2024.

[8] D. Hendrycks et al. Measuring mathematical problem solving with the MATH dataset. In Proceedings of NeurIPS Track on Datasets and Benchmarks, 2021.

[9] D. Phan et al. Humanity's Last Exam. Scale AI, 2025. https://scale.com/hle.

[10] T. Wolf et al. Transformers: State-of-the-art natural language processing. In Proceedings of EMNLP: System Demonstrations, pages 38–45, 2020.

[11] E. R. Tufte. The Visual Display of Quantitative Information. Graphics Press, 2nd edition, 2001.

[12] Render. Render cloud application platform. https://render.com, 2024.